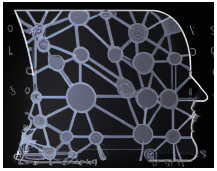
Applied Machine Learning

Feature Engineering 2 - Feature Selection

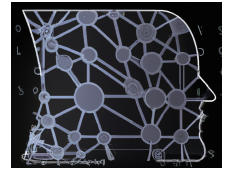


Learning goals

- Reasons for Feature Selection
- Types of Feature Selection

Recap

WHAT IS FEATURE ENGINEERING?



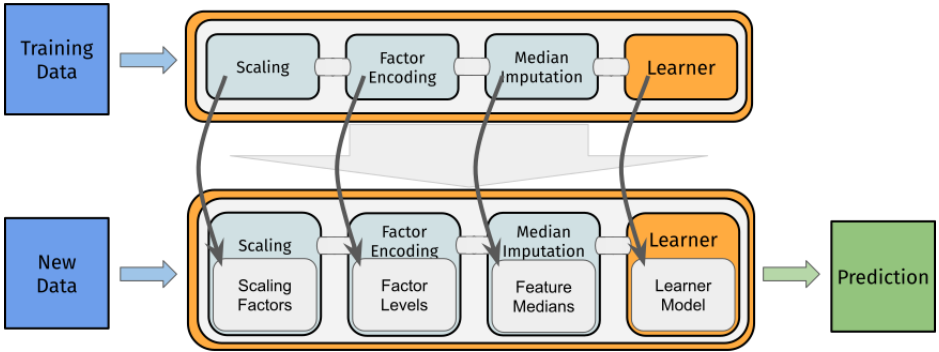
Feature engineering is on the intersection of **data cleaning**, **feature creation** and **feature selection**.

The goal is to solve common difficulties in data science projects, like

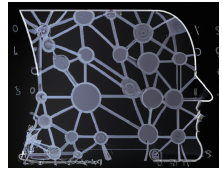
- skewed/weird feature distributions,
- (high cardinality) categorical features,
- functional (temporal) features,
- missing observations,
- high dimensional data,
- ...

and improve model performance (or make the data readable by the model in the first place).

TRAIN-TEST LEAKAGE



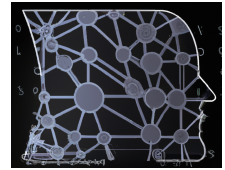
TYPES OF FEATURE ENGINEERING (SO FAR)



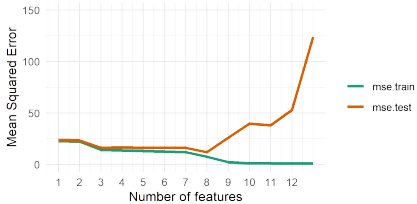
- numerical encoding of categorical features
- feature transformations
- imputation

Feature Selection

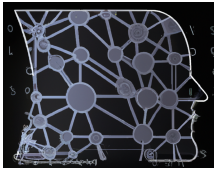
MOTIVATION



- Naive view:
  - More features $\rightarrow$ more information $\rightarrow$ discriminant power $\uparrow$
  - Model is not harmed by irrelevant features since their parameters can simply be estimated as 0.
- In practice, irrelevant and redundant features can "confuse" learners (see **curse of dimensionality**) and worsen performance.
- Example: In linear regression, R^2 is monotonically increasing in p, but adding irrelevant features leads to overfitting (capturing noise).

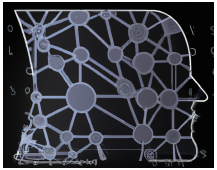


MOTIVATION



- In high-dimensional data sets, we often have prior information that many features are either irrelevant or of low quality
- Having redundant features can cost something during prediction (money or time)
- Many models require $n > p$ data. Thus, we either need to
  - adapt models to high-dimensional data (e.g., regularization)
  - design entirely new procedures for $p > n$ data
  - use filter preprocessing methods from this lecture

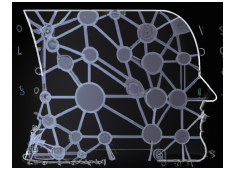
SIZE OF DATASETS



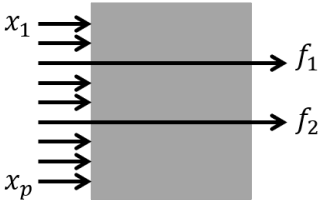
Many new forms of technical measurements and connected data leads to availability of extremely high-dimensional data sets.

- **Classical setting**: Up to around 10^2 features, feature selection might be relevant, but benefits often negligible.
- **Datasets of medium to high dimensionality**: At around 10^2 to 10^3 features, classical approaches can still work well, while principled feature selection helps in many cases.
- **High-dimensional data**: 10^3 to 10^9 or more features. Examples: micro-array / gene expression data and text categorization (bag-of-words features). If we also have few observations, scenario is called $p \gg n$.

FEATURE SELECTION VS. EXTRACTION

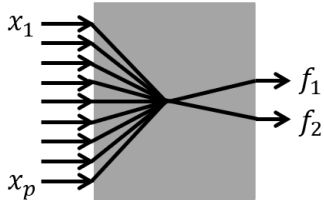


Feature selection



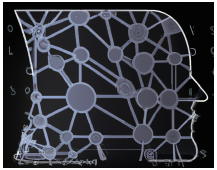
- Creates a subset of original features **x** by selecting $\tilde{p} < p$ features ***f***.
- Retains information on selected individual features.

Feature extraction



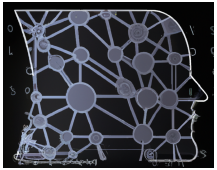
- Maps p features in **x** to $\tilde{p}$ extracted features ***f***.
- Info on individual features can be lost through (non-)linear combination.

REASONS FOR FEATURE SELECTION



- improved predictive performance, since we reduce overfitting on irrelevant features,
- robust models that do not rely on noisy features,
- simpler models that are easier to interpret,
- faster model fitting, e.g. for model updates,
- faster prediction, and
- no need to collect potentially expensive features.

GOALS OF FEATURE SELECTION

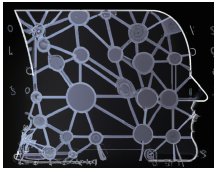


Single Feature Selection Problem: identifying a minimal-size subset of the variables that is optimally predictive for an outcome variable T of interest.

Multiple Feature Selection Problem: Let $\mathcal{S}$ be the solution to the single feature selection problem (called the reference solution). The solution $\mathcal{M}$ to the multiple feature selection problem consists of all minimal-size sets $\mathcal{S}_i \in \mathcal{F}$ that are statistically equivalent to $\mathcal{S}$.

- Important for knowledge discovery (know all possible solutions)
- We will not delve into this any further

TYPES OF FEATURE SELECTION METHODS

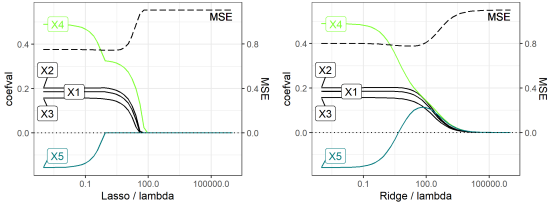


In rest of the chapter, we introduce different types of methods for FS:

- Filters: evaluate relevance of features using statistical properties such as correlation with target variable
- Wrappers: use a model to evaluate subsets of features
- Embedded methods: integrate FS directly into specific model - we look at them in their dedicated chapters (e.g., CART, L_0, L_1)

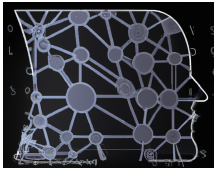
Example: embedded method (Lasso) regularizing model params with $L1$ penalty enables “automatic" feature selection:

$$\mathcal{R}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{R}_{\text{emp}}(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_1 = \sum_{i=1}^{n} \left(y^{(i)} - \boldsymbol{\theta}^\top \mathbf{x}^{(i)}\right)^2 + \lambda \sum_{j=1}^{p} |\theta_j|$$

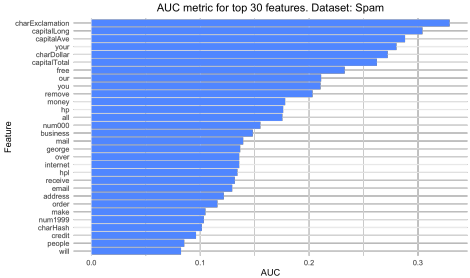


Filtering

INTRODUCTION

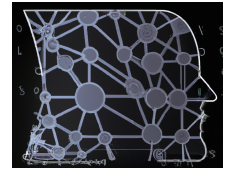


- **Filter methods** construct a measure that quantifies the dependency between features and the target variable.
- They yield a numerical score for each feature x_j, according to which we rank the features.
- They are model-agnostic and can be applied generically.



Exemplary filter score ranking for Spam data.

PEARSON & SPEARMAN CORRELATION



Pearson correlation $r(x_j, y)$:

- For numeric features and targets only.
- Measures linear dependency.
- $r(x_j, y) = \frac{\sum_{i=1}^{n}(x_j^{(i)} - \bar{x}_j)(y^{(i)} - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_j^{(i)} - \bar{x}_j)}\sqrt{(\sum_{i=1}^{n} y^{(i)} - \bar{y})}}, \qquad -1 \leq r \leq 1$

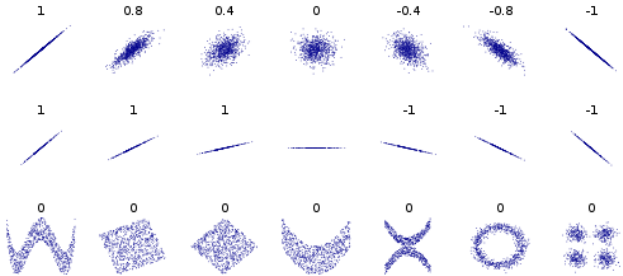
Spearman correlation $r_{SP}(x_j, y)$:

- For features and targets at least on ordinal scale
- Equivalent to Pearson correlation computed on ranks
- Assesses monotonicity of relationship

Use absolute values $|r(x_j, y)|$ for feature ranking:
higher score indicates a higher relevance.

PEARSON & SPEARMAN CORRELATION

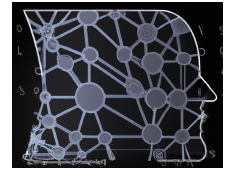
Only **linear** dependency structure, non-linear (non-monotonic) aspects are not captured:



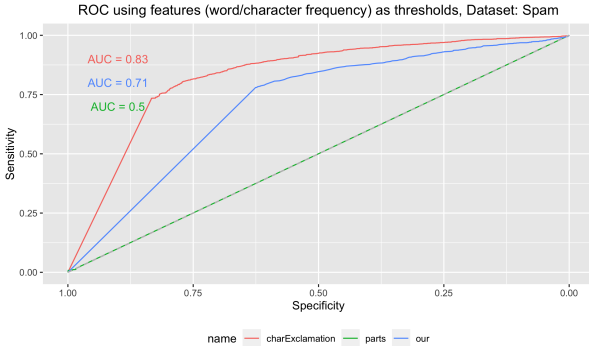
Comparison of Pearson correlation for different dependency structures.

To assess strength of non-linear/non-monotonic dependencies, generalizations such as **distance correlation** can be used.

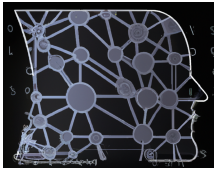
AUC/ROC



- For binary classification with $\mathcal{Y} = \{0, 1\}$ and numeric features
- Classify samples using single feature (with thresholds), compute AUC per feature as proxy for its ability to separate classes
- Features are then ranked; higher AUC scores $\rightarrow$ higher relevance.



FURTHER CRITERIA/METHODS



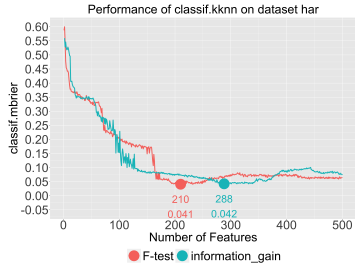
- Classification
  - χ^2-statistic
  - Welch's t-Test
  - F-Test
- Regression & Classification
  - Mutual Information
  - Relief
- Model-based
  - Tree-based feature importance (decision tree, extra trees, random forest)
  - Coefficients of linear model

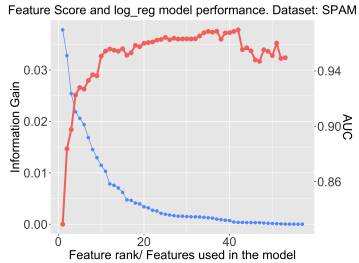
USING FILTER METHODS

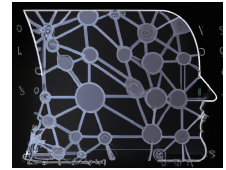
1. Calculate filter score for each feature x_j
2. Rank features according to score values.
3. Choose $\tilde{p}$ best features
4. Train model on $\tilde{p}$ best features.

How to choose $\tilde{p}$?

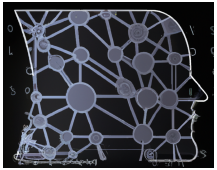
- Could be prescribed by the application
- Eyeball estimation: read from filter plots
- Treat as hyperparameter and tune in a pipeline, based on resampling







ADVANTAGES AND DISADVANTAGES



Advantages

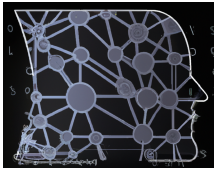
- Fast
- Typically scales well with the number of features p
- Generally interpretable
- Can be used with any inducer

Negative

- Does not take specifics of the inducing algorithm into account
- Redundant features will have similar weights
- Often univariate

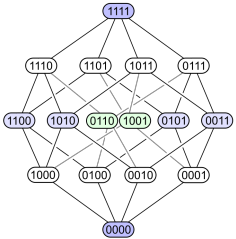
Wrapper

INTRODUCTION



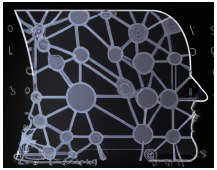
- Wrapper methods emerge from the idea that different sets of features can be optimal for different learners
- Wrapper is a discrete search strategy for S, where objective criterion is test error of learner as function of S. Criterion can also be calculated on train set, approximating test error (AIC, BIC)

$\Rightarrow$ Use the learner to assess the quality of the feature sets



Hasse diagram illustrating search space. Knots are connected if Hamming distance = 1
(Source: Wikipedia)

OBJECTIVE FUNCTION

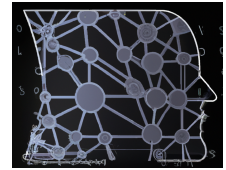


Given p features, **best-subset selection problem** is to find subset $S \subseteq \{1, \dots p\}$ optimizing objective $\Psi : \Omega \to \mathbb{R}$:

$$S^* \in \arg\min_{S \in \Omega} \{\Psi(S)\}$$

- Ω = search space of all feature subsets $S \subseteq \{1, \dots, p\}$. Usually we encode this by bit vectors, i.e., $\Omega = \{0, 1\}^p$ (1 = feat. selected)
- Objective Ψ can be different functions, e.g., AIC/BIC for LM or cross-validated performance of a learner
- Poses a discrete combinatorial optimization problem over search space of size = 2^p, i.e., grows exponentially in p (power set)
- Unfortunately can not be solved efficiently in general (NP hard; see, e.g., Natarajan, 1995)
- Can avoid searching entire space by employing efficient search strategies, traversing search space in a “smart" way

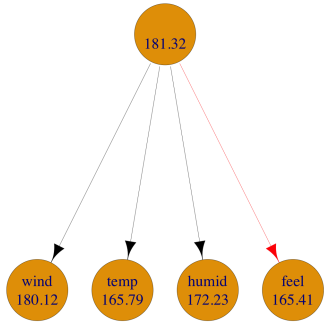
GREEDY FORWARD SEARCH



Let $S \subset \{1, \ldots, p\}$ be subset of feature indices.

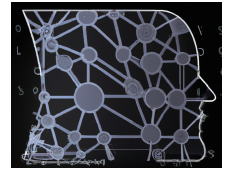
1. Start with the empty feature set $S = \emptyset$
2. For a given set S, generate all $S_j = S \cup \{j\}$ with $j \notin S$.
3. Evaluate the classifier on all S_j and use the best S_j

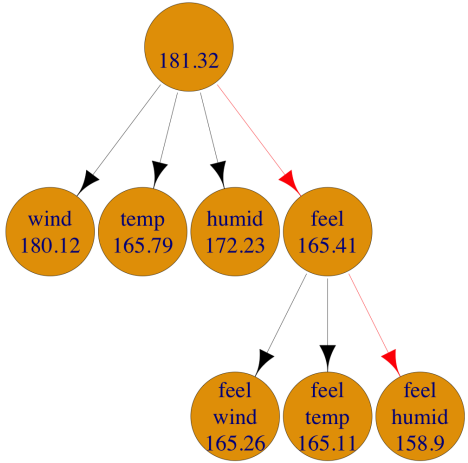
Example GFS on a subset of bike sharing data with features windspeed, temp., humidity and feeling temp. Node value is RMSE.



VISUALIZATION OF GFS

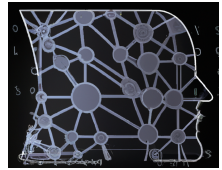
4. Iterate over this procedure

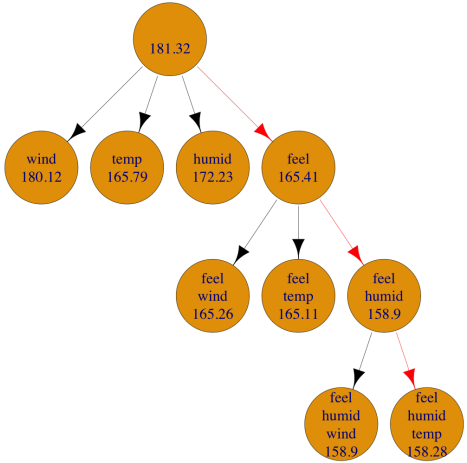




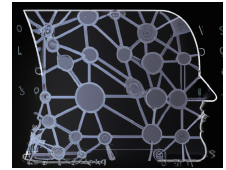
VISUALIZATION OF GFS

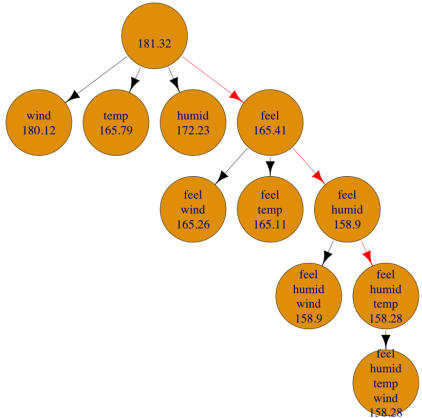
4. Iterate over this procedure





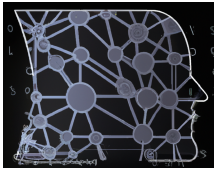
VISUALIZATION OF GFS





5. Terminate if performance does not improve further or max. number of features is used

GREEDY BACKWARD SEARCH

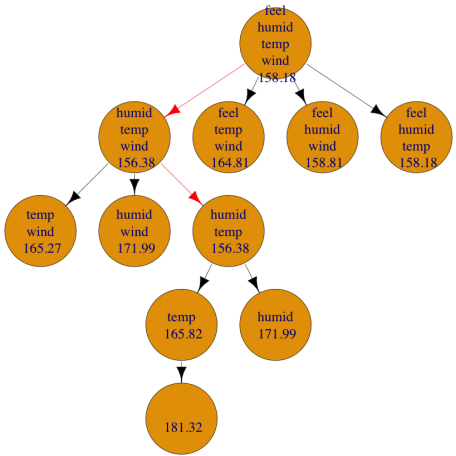


- Start with the full index set of features $S = \{1, \ldots, p\}$.
- For a given set S generate all $S_j = S \setminus \{j\}$ with $j \in S$.
- Evaluate the classifier on all S_j and use the best S_j.
- Iterate over this procedure.
- Terminate if:
  - the performance drops drastically, or
  - falls below given threshold.

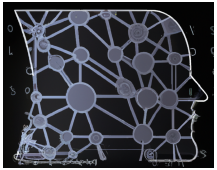
- GFS is much faster and generates sparser feature selections
- GBS much more costly and slower, but sometimes slightly better.

VISUALIZATION OF GBS

Example Greedy Backward Search on bike sharing data



ADVANTAGES AND DISADVANTAGES



Advantages

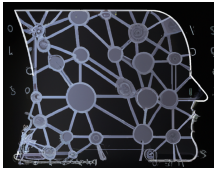
- Inducer-agnostic
- Any performance measure can be used
- Optimizes the desired performance measure directly

Disadvantages

- Expensive
- Does not scale well with the number of features
- Does (in general) not use additional info about model structure
- Nested resampling becomes necessary

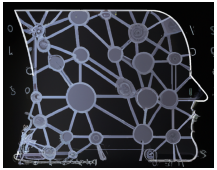
Further Selection Techniques

EMBEDDED FEATURE SELECTION



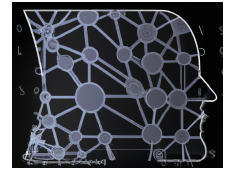
- Select features during the learning process
- Explicitly – via regularization
  - LASSO/L1
  - Requires a solver that can set weights to zero
- Implicitly – by construction of the algorithm
  - Decision-tree based algorithms
  - Can ignore irrelevant features by not selecting them (no guarantee, though)

DOMAIN KNOWLEDGE

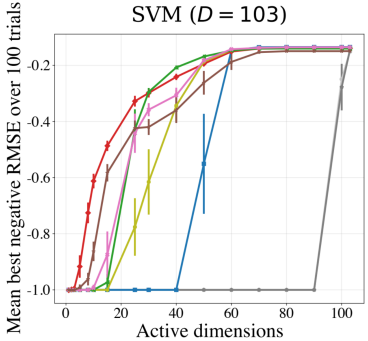


- ID columns
  - Often ordered, can leak time information
  - If they identify a person etc, replace by features describing that person
  - Contains no information that can generalize
- Duplicate features
  - Cause issues for (linear) models
  - Slow down learning
  - Reduce model interpretability

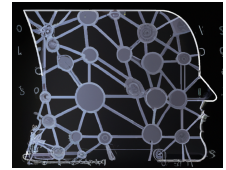
MULTI-OBJECTIVE OPTIMIZATION OF FEATURES



- Optimize two competing Objectives:
  1. goodness-of-fit
  2. number of variable
- Use global optimization algorithm
  - Evolutionary Algorithm
  - Sparse Bayesian optimization



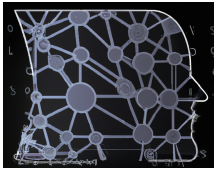
BORUTA (1)



- All-relevant feature selection
  Goal: find all features that have affect the prediction
  $\rightarrow$ Can include redundant features
- Idea: use *shadow variables* to contrast existing features against
- Method: Extend a feature scoring to an iterative testing mechanism

Boruta for those in a Hurry by Miron B. Krusa

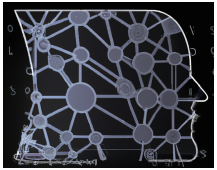
BORUTA (2)



1. Create a copy of each feature, shuffle these copies
2. Fit a random forest on the new dataset
3. An attribute is deemed important, if its feature importance is higher than the maximal importance of all randomised attributes
4. Repeat steps 1-3 for N iterations
5. Execute SHT with the null hypothesis that the importance of a feature is equal to the maximal importance of all randomised attributes
6. Repeat steps 1-5 until the feature set is stable

Wrap-up

TAKEAWAYS



- There can be multiple goals in feature selection:
  - Single Feature Selection
  - Multiple Feature Selection
  - All-relevant Feature Selection
- Important step for good prediction quality and fast models
- Different feature selection methods have different goals and use different mechanisms $\rightarrow$ model selection problem